Caching and Persistence in Spark

In this train, we'll delve into caching and persistence techniques within Apache Spark. We'll focus on how to optimize Spark's performance by retaining data in memory or on disk, reducing the need for repeated computations.

Learning Objectives

By the end of this train, you should be able to:

- Understand and apply caching and persistence of data.

Understanding Caching

way that you can boost the performance of your application is by caching or persisting data that you use frequently. Spark has two APIs to persist or cache data: `cache()` and `persist()`. `cache()` allows you to store as many partitions in memory as is available. Thus, if you have eight partitions and you have space to partition six of them, you will be able to cache those partitions in memory. You cannot cache partial partitions. Partitions not cached will have to recompute when running your application, which will lead to an application slowdown. Let's see this in practice. We create a dummy dataset and apply some complex transformations to the dataset.

```
df = spark.range(1*1000000)
df = df.withColumn('tenth', F.col('id') ** 10)
```

Let's cache the dataset. However, it is only cached when we perform a subsequent action.

```
df.cache()
```

Output:

```
DataFrame[id: bigint, tenth: double]
```

The `count()` method below will invoke an action and, subsequently, cache the result. We use the `%%time` cell magic to time one run of the whole cell.

```
%%time

df.count()
```

Output:

```
1000000
```

We can now run `count()` again, and check if the result is returned a bit quicker.

```
%%time

df.count()
```

Output:

```
1000000
```

Caching the data before performing an action on the DataFrame can result in a large performance improvement. In the example above, the data is cached once the `df.count()` action is performed on the dataset, and each subsequent `df.count()` action will have performance improvements.

Caching Data via Persistence

Another way of caching data is by using the `persist()` method.

Persist provides slightly more control over how the data are persisted, as you can specify the storage level at which the data should be persisted. Storage levels available are summarized in the table below.

StorageLevel	Description
MEMORY_ONLY	Data is stored directly as objects and stored only in memory.
MEMORY_ONLY_SER	Data is serialized as compact byte array representation and stored only in memory. To use it, it has to be deserialized at a cost.
MEMORY_AND_DISK	Data is stored directly as objects in memory, but if there's insufficient memory the rest is serialized and stored on disk.
DISK_ONLY	Data is serialized and stored on disk.
OFF_HEAP	Data is stored off-heap. Off-heap memory is used in Spark for storage and query execution; see "Configuring Spark executors' memory and the shuffle service" on page 178.
MEMORY_AND_DISK_SER	Like MEMORY_AND_DISK, but data is serialized when stored in memory. (Data is always serialized when stored on disk.)

Storage levels available.

```
df = spark.range(1*1000000)
df = df.withColumn('tenth', F.col('id') ** 10)
```

```
from pyspark.storagelevel import StorageLevel

df.persist(StorageLevel.DISK_ONLY)
```

Output

```
DataFrame[id: bigint, tenth: double]
```

```
%%time

df.count()
```

```
CPU times: user 1.31 ms, sys: 1.95 ms, total: 3.26 ms
Wall time: 101 ms
```

```
%%time

df.count()
```

```
CPU times: user 1.38 ms, sys: 2.3 ms, total: 3.68 ms
Wall time: 104 ms
```

Hopefully, you'll notice a massive speed-up. This, again, will depend on the machine that you are using. If you have a solid-state hard drive, it will be a lot faster than caching to a hard disk drive.

Conclusion

Caching and persistence in Spark are techniques used to optimize performance by storing frequently used data in memory or on disk to avoid redundant computations.

▼Key Takeaways ✖

- **Takeaway 1:** Spark provides `cache()` for in-memory storage (as much as available) and `persist()` for more control over storage levels, including disk-only options.
- **Takeaway 2:** Data is only cached or persisted after an action is performed on the DataFrame.
- **Takeaway 3:** Caching or persisting data can significantly improve the performance of subsequent operations on the same data.